Agent 开发面试题与准备建议

整理了 23 场 Agent 开发、AI 应用平台和后端岗位的面试问题。前面按公司、轮次列题，后面汇总编码题和准备建议。

面试概览

公司 / 业务	收录轮次	主要内容
字节跳动	一、二、三面	Runtime、故障恢复、权限、可观测性、现场编码
千问	一、二、三面	Harness 边界、能力扩展、Memory、上下文、评测
淘天	一、二面	Agent 与 Workflow 取舍、运行生命周期、安全、Plan-and-Execute
小红书	方向1三轮；方向2两轮	多 Agent、RAG、记忆、强约束流程、分布式可靠性
美团	一面	Skill、效果归因、能力边界、自研系统的实际价值
拼多多	一面	记忆治理、沙箱、跨租户隔离、多 Agent 恢复、安全
腾讯	一面	上下文、多模态、SSE 重连、异步长任务、人工交互
蚂蚁	一、二面	后端与数据链路、RAG 排障、运行内核、工程协作
百度	一、二、三面	Workflow、RAG、Python 并发、流式恢复、评测指标
深信服	一面	应用落地、Agent 架构、Workflow 与 ReAct、能力边界
同花顺	一面	Skill 与 MCP、云端 Agent、资源隔离、Python / Java 编码

一、字节跳动

一面

1. 为什么选择 LangGraph？与其他 Agent 框架相比，看中了哪些能力？
2. ReAct 循环在什么情况下会迟迟不结束？
3. 追问：如何限制循环次数、上下文规模和工具调用范围？
4. Tool Calling 的协议怎么设计？模型看到的工具描述与实际执行实现如何对应？
5. Agent 执行到一半进程崩溃，恢复后怎样找到正确的执行位置？
6. 追问：如果外部工具已经执行过了，恢复时怎么避免重复调用、重复产生副作用？
7. 面对理解需求、检索资料、生成报告等多步骤任务，会选择单 Agent 加工具，还是多 Agent？
8. 追问：Manager-Worker 这样的架构适合什么情况？与单 Agent 的取舍是什么？
9. Agent 运行时记录了哪些 Metrics、Trace 和 Log？
10. RAG 使用什么向量存储？
11. 检索返回了很多候选片段，但上下文空间有限，如何排序、压缩和组装？
12. Agent 如何部署？逻辑上的 Agent 与进程、Worker 是什么关系？
13. 追问：每个 Agent 都需要一个独立运行实例吗？
14. 后端平台与 AI 平台如何协作，哪些能力可以复用？
15. 如何统一接入不同的模型 Provider？
16. 追问：不同 Provider 的流式协议、限流、降级和成本统计怎么处理？
17. **【算法题】** Two Sum 类问题，如何通过一次遍历找到目标组合？题面信息不完整，不还原额外约束。

本轮小结

范围比较广，从 LangGraph 选型、ReAct 收敛、Tool Calling，一直问到多 Agent 取舍、RAG、可观测性和模型网关，最后还有一道基础算法题。问题虽然多，但都围绕同一件事：一个 Agent 应用怎样变成能部署、能持续运行、出问题也能排查的服务。

其中比较深入的是循环失控和故障恢复。前者要同时考虑轮次、上下文和工具权限，后者要分清恢复运行状态与避免外部操作重复执行，不能只回答“有限制”或“有 Checkpoint”。模型网关题还要求覆盖多 Provider 之外的流式、限流、降级和成本。准备这类面试，可以先梳理完整调用链，再逐项补上失败处理；算法部分则另看独立编码和复杂度分析。

二面

1. 为什么考虑换工作？目前的工作地点和职级情况如何？
2. AI 应用平台最初为什么立项，面向什么用户？
3. 追问：现在有哪些真实使用场景，用户实际怎么使用？
4. 用户在平台上创建的应用如何发布、嵌入或集成到已有系统？
5. 如果从平台研发转向客户现场交付，可以沉淀哪些方法和可复用资产？
6. 通用运行内核、业务扩展与服务入口各负责什么？

7. 业务扩展由谁开发？客户最终买到的是产品、服务还是一套定制应用？
8. 云端部署与私有化部署分别怎么做？
9. 追问：客户的数据不能出内网时，模型部署和调用链路如何处理？
10. 工具在平台中怎样抽象？能否讲清一次工具调用的链路？
11. Workflow 的工具节点和 ReAct 的 Tool Calling 能否复用同一套执行能力？
12. 运行时对 LangGraph 的依赖有多深？
13. 追问：Python 性能是否会成为瓶颈？
14. 用户身份和角色如何传递到运行时？工具和 Skill 的可见范围、调用权限怎样控制？
15. 日志、Trace 和指标存在哪里？不同人员如何获得查看和排障权限？
16. 排障效率的提升具体怎么测量？
17. 追问：比较基线和统计范围是什么，如何证明是这次改动带来的提升？
18. 是否考虑过建设统一控制面？当前为什么做或不做？
19. 你在项目中具体负责哪些部分？
20. 行业 Agent 实际解决了什么问题，交付物是什么？
21. 托管式 Agent Runtime 是否可行，客户会因为什么选择它？
22. **【算法题】** 将两个有序数组原地合并到具有预留空间的第一个数组中。

本轮小结

相比一面，这轮更关注平台怎样被客户使用：应用如何交付，云上与私有化部署有什么不同，业务扩展怎样复用运行时，身份和权限怎样传到工具层。日志授权、统一控制面和托管服务的讨论，也都与交付后的运维有关。

一个明显的侧重点是核对设计与实际工作是否对应。比如工具分层解决了什么问题，排障效率提升是怎样测量的，建设控制面是否值得投入。准备时除了架构图，还应有部署、授权、排障的具体过程，以及指标的比较基线。合并有序数组是单独的现场算法环节，原地修改、指针移动和数组耗尽后的处理都需要独立练习。

三面

1. AI 应用平台的整体架构怎么设计，采用了哪些技术？
2. 为什么选择 LangGraph？
3. DSL 如何定义，又如何转换为执行图？
4. LangGraph 使用过程中遇到过什么问题？
5. 追问：是否做过性能测试，系统容量如何？
6. 平台质量怎么衡量，主要看哪些指标？
7. 平台后续如何演进？业务 Agent 是否直接建立在原平台之上？
8. LangGraph 之外，团队自行实现了哪些能力？
9. 运行时怎样划分职责，通用执行层与业务扩展如何分工？
10. **【工程编码题】** 写出 ReAct Loop 伪代码，并说明如何通过 Hook 接入 Skill、Memory 等能力。
11. Memory 在什么时候检索、更新和保存？
12. 工具调用超时怎么处理？
13. 页面关闭后，后台任务如何继续执行？

14. 多个 Worker 怎么避免执行冲突?

15. 如何看待 Agent 与 Harness 的后续演进?

16. 目前的职级是什么?

本轮小结

这一轮把平台架构、性能指标和运行机制放在一起核对，并要求现场写 ReAct Loop 伪代码。相比前两轮，重点更接近“能否把自己描述的设计展开”：循环如何组织，Hook 放在哪，Skill 和 Memory 何时介入，工具异常怎么返回给后续步骤。

后续又从正常流程追到工具超时、页面关闭、多个 Worker 冲突，说明只写出模型与工具交替调用还不够。准备时可以先写一个完整的小循环，再解释记忆、错误处理和恢复分别加在哪里。性能部分则要分清部署配置与实际容量，技术演进问题也最好联系具体约束来讲，不必一次铺开很多架构名词。

二、千问

一面

1. 介绍一下当前工作和比较擅长的方向。
2. 对工作地点是否有要求，能否接受异地机会?
3. AI 应用平台与业务 Agent 项目是什么关系?
4. 你在其中具体承担哪些工作?
5. 为什么需要单独建设 Harness，它要解决什么问题?
6. 企业 Agent 除了 ReAct、Tool Calling 和上下文管理，还需要补哪些能力?
7. 通用运行内核、业务扩展与服务入口如何划分?
8. 业务扩展模块具体包含什么?
9. 这些模块如何注册、管理版本和升级?
10. 运行内核如何与具体业务实现解耦?
11. Memory 策略应该由业务决定，还是固定在运行内核中?
12. 能力声明文件需要描述哪些信息?
13. Tool、Skill、Artifact 和 Hook 如何组织?
14. Workflow 与 ReAct 如何配合?
15. 业务扩展到什么程度应当修改内核，边界在哪里?
16. 目前有多少类业务扩展，是否经过不同场景验证?
17. 多人同时开发不同业务能力时，怎样协作和集成?
18. 不同业务独立部署，还是共享运行时?
19. 追问：共享时怎么做隔离?
20. Skill 如何路由和选择?
21. 用户怎样发现和使用已有 Skill?
22. Skill 是代码实现还是配置定义?
23. 是否提供沙箱，主要解决什么问题?
24. 沙箱带来的延迟与安全收益怎么权衡?
25. Serverless 与临时创建 Kubernetes Pod 的方式有什么差别?
26. 目前的用户量和并发规模如何?
27. 实时交互与后台长任务如何安排 Worker?
28. 你在平台项目中负责哪些模块?
29. 平台的实际使用规模怎样，是否已有线上用户?
30. 应用如何发布和集成到外部系统?
31. 底层执行引擎是什么?
32. 画布 DSL 如何转换为执行图?
33. 是否比较过 AgentScope、pi-agent 等框架，各自有什么特点?

34. ToB 与 ToC Agent 的主要区别是什么?
35. 面向 ToC 时，Memory 应该怎样设计?
36. 工具越来越多，会带来哪些上下文和选择噪声，如何处理?
37. Skill 规则与 Memory 冲突时怎么办?
38. Python 和 Java 分别使用到什么程度?
39. 如果团队使用另一种语言，准备怎样适应?

本轮小结

这轮主要围绕企业级 Harness 的组成和扩展方式展开，覆盖业务解耦、能力注册与版本、Skill 路由、Workflow 组合、共享运行时、沙箱以及画布编译。还把讨论从企业应用延伸到个人 Agent，追问工具数量和 Memory 冲突带来的变化。

比较有特点的是问题会顺着架构逐层往下走：说有业务扩展，就问扩展里放什么、怎么注册、如何升级；说可以共享 Runtime，就问权限和隔离；说能调用 Skill，就问请求怎样找到它。准备时适合用一个新增能力接入的例子讲清全过程，再区分哪些机制通用、哪些策略随业务变化，避免仅用分层名称回答。

二面

1. 请做一下自我介绍。
2. 最近在读哪些 Agent 框架?
3. 是否看过 Coding Agent 的实现，重点研究了哪些部分?
4. 怎样判断一个 Harness 好不好?
5. 自己的运行时实现了什么，能力边界在哪里?
6. 通用运行内核、业务扩展和服务入口怎样分层?
7. Tool、Skill 和 Memory 的接口分别如何设计?
8. 运行时的输入和输出是什么?
9. 多个业务并行扩展时，怎样避免互相影响?
10. 如何评估自研系统与开源方案的差异?
11. 追问：是否真正用相同任务对比过通用 Agent?
12. 测试用例、Ground Truth 和 A/B 对比具体怎么做?
13. 当前使用哪些模型?
14. 是否做过模型后训练?
15. 框架给开发者提供什么接口，开发者需要写哪些代码?
16. 与 AgentScope 相比，有什么不同?
17. 是否了解托管式 Agent 服务?
18. 目前有多少真实用户，实际使用情况如何?
19. 开发中遇到的最大挑战是什么?
20. 较重的 Workflow 如何与 ReAct 主循环结合?
21. Sub-agent 如何定义和启动?
22. 追问：子 Agent 是提前声明的，还是运行过程中动态创建的?

23. 为什么考虑离开当前公司?
24. 为什么选择这个方向和这家公司?
25. 如何看待当前公司的优势与不足?
26. 后续职业规划是什么?

本轮小结

范围包括 Harness 的质量标准、Runtime 输入输出、接口与扩展、Workflow 和 Sub-agent，以及模型、后训练和框架产品形态。与一面相比，这轮不只是确认系统由哪些部分组成，而是进一步问这些设计能带来什么效果。

自研 Harness 与通用 Agent 的对比是值得重点准备的一段：有没有真实比较过，用什么用例和参考结果，怎样使用 A/B 测试，结论在什么场景下成立。Runtime 的输入输出也被单独追问，需要比“接收请求、返回答案”更具体。源码阅读和技术趋势可以展示视野，但仍要回到实现与验证，不能用新框架的名称代替效果依据。

三面

1. 请做一下自我介绍。
2. 入职当前公司时是什么级别，现在是什么级别?
3. 是否有晋升经历？如有，最近一次是什么时候?
4. 追问：如果有晋升经历，主要凭借哪些工作和贡献?
5. 如果有晋升和转向 AI 研发的经历，两者的时间先后关系是什么?
6. 介绍一下当前业务 Agent 项目。
7. FDE 在实际工作中意味着什么?
8. 项目团队有哪些角色，怎样分工?
9. 是否所有研发人员都参与运行内核开发?
10. 业务 Agent 是否建立在原有应用平台之上?
11. 你具体负责哪部分?
12. 架构方案由谁决定，你参与了哪些设计决策?
13. 为什么采用运行内核、业务扩展和服务入口的分层方式?
14. 运行内核内部包含哪些模块?
15. Workflow 和 ReAct 如何结合?
16. 除此之外，还有哪些比较有特点的设计?
17. 为什么把 Memory 实现放在运行内核之外?
18. 记忆接口如何定义?
19. 追问：具体实现由谁提供，又怎样注入运行时?
20. 上下文工程做了哪些工作?
21. Token 预算如何分配，什么时候触发压缩?
22. 工具结果和历史消息分别怎么压缩?
23. 这些设计中，哪些已经做过实际验证?
24. 平时怎样追踪 Agent 领域的变化?

25. 最近观察到哪些值得关注的架构趋势?

26. 下一阶段的职业发展考虑是什么?

27. 工作地点上有哪些考虑?

本轮小结

技术部分集中在架构方案的来由、运行内核的组织、Workflow 与 ReAct 的结合，以及 Memory 接口和上下文压缩。与二面对效果的关注相比，这轮更多是在追问设计理由：为什么选择这种分层，为什么记忆实现不直接写死在内核里，接口由谁实现、怎样接入。

上下文部分要求说清处理对象和触发条件，包括 Token Budget、工具结果、历史消息以及压缩做法。准备时可以拿一个接口或一项压缩策略，解释原先的问题、选过哪些方案、最后为何这样做。对技术变化的讨论，也适合说明新能力会影响现有系统的哪一部分，而不只是介绍最近关注了什么。

三、淘天

一面

1. 请做一下自我介绍。
2. 单 Agent、多 Agent 和 Workflow 分别适合什么场景?
3. 它们的核心区别是什么?
4. 任务复杂，就一定应该用多 Agent 吗?
5. 追问：什么条件下才值得引入多 Agent?
6. 为什么将 Workflow 封装为 Skill?
7. 一次用户请求从进入到结束，生命周期是什么样的?
8. 运行内核、业务扩展和服务入口分别负责什么?
9. 一次完整任务与其中一轮模型推理、工具调用是什么关系?
10. Skill 如何被选中，并进入上下文?
11. 工具调用什么时候并行，什么时候串行?
12. 为什么选择 LangGraph?
13. Agent 效果从哪些维度评估?
14. 测试用例和参考执行轨迹怎么构建?
15. LLM-as-a-Judge 具体评价哪些内容?
16. 怎样把评测接入发布和回归流程?
17. 如果上线后效果下降，怎么定位和回滚?
18. 如何防范 Prompt Injection?
19. 沙箱承担哪些安全职责?
20. Agent 需要访问敏感数据时，怎样控制权限?
21. 上下文接近模型窗口上限时怎么办?
22. 平时如何保持上下文干净，避免无关信息不断积累?
23. 工具返回过长，或连续调用工具产生大量记录时，怎么处理?
24. 追问：压缩时哪些信息必须保留?
25. 业务场景中的长期记忆如何组织?
26. 新旧记忆冲突时怎么处理?
27. **【工程编码题】** 实现一个简化的 Plan-and-Execute Agent：查询两个城市的天气，并计算温度差。
28. Planner、Executor 和 Summarizer 各负责什么?
29. 步骤怎样表示，后续步骤如何引用前序结果?
30. 怎样区分调用工具的步骤和调用模型的步骤?
31. 工具注册表如何设计?
32. 目前其他求职流程进展如何?
33. 对工作地点有什么考虑?

34. 为什么考虑换工作?

本轮小结

前半段覆盖单 Agent、多 Agent 与 Workflow 的选型，接着看任务生命周期、工具调度、评测回归、Prompt Injection、权限和上下文。后半段要求实现一个 Plan-and-Execute Agent，属于与岗位直接相关的工程编码，而不是常规算法题。

选型问题会追问“复杂任务为什么一定需要多 Agent”，不能只按任务复杂度分类。编码则把前面的讨论变得具体：计划是什么数据结构，步骤怎样引用已有结果，哪些步骤调用工具、哪些交给模型，最后怎样汇总。准备时最好能完成一个小而完整的实现，再解释安全校验、错误和预算加在哪里；只会画 Planner、Executor、Summarizer 三个方框还不够。

二面

1. 请做一下自我介绍。
2. AI 应用平台最初要解决什么问题，为什么建设这个平台?
3. 与 Dify、Coze、Palantir 等方案相比，借鉴了什么，又有什么不同?
4. 是否阅读过相关开源项目的源码?
5. Workflow 节点非常多时，怎样处理复杂度?
6. 是否支持通过自然语言生成 Workflow?
7. 为什么不让模型直接生成最终 DSL?
8. 生成后怎样校验节点、参数和连线?
9. 能否自动执行测试，并根据错误修复生成结果?
10. 工具数量很多时，如何组织和选择?
11. 如果是面向 ToC 的 Agent，工具信息如何进入上下文?
12. 如何通过作用域和渐进式披露限制当前可见能力?
13. 工具描述和历史消息过长时如何压缩?
14. 完整的 RAG 链路怎么设计?
15. 不同文件类型分别如何处理?
16. PPL 切分是怎么做的?
17. BM25、向量检索、RRF、MMR 和 Rerank 如何组合?
18. 如何评估检索和回答效果?
19. 如果生成答案和评价答案都依赖同类模型，怎样看待评测偏差?
20. 模型生成的评测集可信吗，怎样验证?
21. 在已有平台之外，为什么还需要独立的业务 Harness?
22. 运行内核、业务扩展与服务入口怎样分工?
23. 这套运行时能否迁移到其他行业?
24. 不同行业的业务指标不同，通用能力和业务评价怎么分开?
25. 页面关闭或服务重启后，任务如何恢复?
26. 如何认领任务，避免重复执行?

27. 追问：如果工具会产生外部副作用，怎样处理重复调用?
28. 从业务模板演进到通用运行时，需要做哪些调整?
29. 这套设计迁移到 ToC 场景，会遇到什么问题?
30. 为什么考虑离开当前公司?
31. 其他求职流程进展如何?

本轮小结

这轮更偏平台演进和方案设计。范围包括节点数量与工具规模治理、自然语言生成 Workflow、RAG 处理链路、评测数据可信度，以及跨业务复用和故障恢复。它把已有系统放到新需求里，观察设计是否还能成立。

自然语言生成 Workflow 是较完整的设计题：不止问模型怎样生成 DSL，还问类型、参数和连边怎样校验，生成后怎样测试和修复。RAG 部分也没有止于召回与重排，而是继续质疑模型生成题目、模型做 Judge 的偏差。准备时应把生成、校验、验证和反馈连起来讲，并能解释工具增多、业务变化或 Worker 重启后会出现什么问题。

四、小红书

方向1一面

1. 平台的定位是什么，核心能力有哪些?
2. 客户实际怎样使用这个平台?
3. 同时支持 Workflow 和自主 Agent 吗？应用如何发布?
4. 平台与现有产品相比，有什么差异化?
5. 为什么用 LangGraph，而不是沿用原有后端语言自行实现?
6. 多 Agent 采用什么协作模式?
7. Planner 如何选择和路由到具体 Agent?
8. 多个 Agent 能力相似时，会不会选错?
9. 追问：路由是否正确，怎样评估?
10. Agent 之间怎样传递上下文?
11. 如果并行执行并写入共享状态，怎么处理冲突?
12. 检索效果用什么指标衡量?
13. 评测数据集怎样构建，预期召回结果由谁确定?
14. 为什么从单一检索演进到混合检索，再加入重排?
15. Query Rewrite 具体做了什么?
16. 追问：提升怎么测出来的，有没有失败案例?
17. 多模态文档如何解析和切分，对检索有什么影响?
18. PPL 切分的原理是什么?
19. 用什么模型计算，成本如何?
20. 切分阈值是固定的还是动态的?
21. 长对话中出现偏离当前任务的消息，怎样处理边界?
22. 短期上下文与长期记忆如何区分?
23. 长期记忆怎样更新?
24. Memory 与固定 Workflow 的关系是什么?
25. 是否支持 Skill？Memory 与 Skill 的规则冲突时怎么办?

本轮小结

这一轮的主要范围是多 Agent、RAG 和 Memory，平台定位与框架选型作为背景。多 Agent 追到职责相近时如何路由、共享上下文如何并发写入；RAG 追到查询改写的收益与失败、PPL 切分的阈值和成本；Memory 则追到长期偏好与 Skill 规则冲突。

这些问题的共同点是，都在检查正常流程之外的偏差怎样被发现和处理。回答“用了 Planner”“做了语义切分”之后，还需要解释为什么会分错、切错或记错，以及如何用数据验证改动。准备时可以分别整理一个任务路由、查询改写和记忆冲突的例子，讲清输入、错误表现、处理方法及其限制。

方向1二面

1. 请做一下自我介绍。
2. 平台的整体架构和核心模块是什么?
3. 画布 DSL 如何转换成 LangGraph 图?
4. Workflow 与 ReAct 如何复用底层能力?
5. 与 OpenClaw 这类 Agent 相比，有什么区别?
6. 长期记忆和个人工作空间怎样设计?
7. 业务数据平台中的数据、逻辑和动作如何接入 Agent?
8. 业务本体由平台统一建设，还是由具体项目提供?
9. 知识图谱怎样查询，查询结果如何用于任务?
10. RAG 是固定执行链路，还是作为 Tool 让模型决定是否调用?
11. 工具返回内容很长时，如何裁剪?
12. 使用了哪些文档切分策略，各自适合什么情况?
13. PDF、PPT、Excel 分别如何处理?
14. 追问：图片和表格怎样解析?
15. 切分粒度与召回效果之间是什么关系?
16. 混合检索和重排具体怎么组合?
17. 为什么没有采用 MMR?
18. Workflow 作为 Skill，究竟是什么意思?
19. Skill、Tool、Workflow 与通用运行内核怎样解耦?
20. 当前职级和过去的绩效情况如何?

本轮小结

范围以平台架构和 RAG 为主：DSL 怎样进入执行引擎，Workflow 与 ReAct 如何复用能力，业务实体和 Action 怎样暴露给 Agent，以及不同文档的解析、切分和检索。个人工作空间与长期记忆的提问，则用来比较业务平台和通用 Harness 的差异。

值得留意的是几组看起来相近、实际执行方式不同的概念：RAG 是固定节点还是自主调用的工具，Skill 是指引还是可执行 Workflow，底层能力如何被不同入口复用。检索部分还会追问 Chunk 粒度、结果过长和 MMR 的取舍。准备时适合画出这几条不同调用路径，再解释为什么某一场景选其中一种。

方向1三面

1. 平台主要解决什么问题，核心指标是什么?
2. 主要部署在公有云还是客户私有环境?
3. 当前客户、用户以及 Agent / Workflow 的使用规模如何?
4. 整体架构怎么设计?
5. 为什么选 LangGraph，与其他框架比较过吗?
6. DSL 从加载、编译到执行，完整过程是什么?

7. 节点类型如何组织，不同能力怎么接入?
8. 与 Dify、Coze 相比有什么不同?
9. Workflow 与 ReAct 怎样共用执行能力?
10. Agent 能否作为一个 Workflow 节点?
11. Worker、Token 和模型资源如何管理?
12. 项目最大的挑战是什么，你具体贡献了什么?
13. 接入 API、MCP、知识库和业务数据分别需要多少开发工作?
14. 为什么引入 Kafka，同步的是什么数据?
15. 运行时如何跨客户复用?
16. 业务扩展模块里包含哪些内容?
17. 上下文如何分层，各层分别放什么?
18. 什么时候触发压缩，怎样保留关键内容?
19. 当前能力处于 PoC 还是生产使用阶段?
20. PoC 验证和最终交付分别由谁负责?
21. 长期记忆如何分类、提取和更新?
22. 多 Agent 如何进行委派、隔离和任务交接?
23. 当前职级与入职时相比有什么变化?
24. 为什么考虑换工作，又为什么选择这个方向?

本轮小结

这一轮从平台目标和指标开始，覆盖真实使用、部署与并发、模型资源、系统接入、上下文分层、长期记忆和多 Agent 协作。相比前两轮较细的机制讨论，它更关注平台整体是否能被使用、能承担多少工作，以及交付新场景需要付出什么成本。

重点是把架构与实际运行对应起来：Worker、模型 API 和 Token 各自限制什么，上下文分层后怎样更新，客户接入为什么能更快，验证阶段与生产使用又有什么区别。准备时应分开说明用户量、任务量、并发和资源配置，并能解释自己的统计口径；仅列出平台支持哪些功能，不足以回答这些问题。

方向2一面

1. 请做一下自我介绍。
2. 平台是基于 LangGraph 建设的吗?
3. 在框架之上，自己补充了哪些能力?
4. 节点和 DSL 怎样转换为可执行图?
5. 文件处理、RAG 和评测能力分别做了哪些扩展?
6. 为什么不直接使用 Dify?
7. 平台的定位和能力边界与已有产品有什么不同?
8. 目前有哪些真实业务在使用?
9. 平台面向 ToB 还是 ToC?
10. Agent 生态变化很快，这些变化对平台有什么影响?

11. 平台更偏用于配置和管理的管理台，还是直接承接在线业务流量的运行台?
12. 配置控制面与运行执行面如何划分?
13. 开放自治 Agent 与固定 Workflow 应怎样取舍?
14. 哪些具体业务适合采用 Agent?
15. 公司内部是否已有相关 SDK 或托管产品，为什么仍需要建设这套能力?
16. 如果模型跳过权限判断或审批节点，仅靠 Skill 描述能否保证强约束流程?
17. 必需的工具没有调用、必需的产物没有生成时，怎样判断任务是否完成?
18. 追问：运行结束时校验不通过，后续怎么处理，是否计为成功?
19. 对工作地点是否有要求?

本轮小结

这一轮以平台定位和技术选型为主，先问在 LangGraph 上补了什么、为什么不用现成平台，再讨论画布产品与开放式 Agent 的关系。管理台与运行台的区分也很重要：前者偏配置和管理，后者直接承担任务执行，两者面对的要求不同。

侧重点不在证明某种技术更先进，而在说明场景需要什么。流程明确、规则必须遵守的任务，与需要探索路径的任务，不宜采用完全相同的控制方式。后半段交流还讨论了 Agent 生成或修改 Workflow 的可能性。准备时可以从配置、发布和执行的职责讲起，说明画布与 Agent 如何配合，避免把技术演进简单说成新方案替代旧方案。

方向2二面

1. 请做一下自我介绍。
2. 挑一个最有代表性的项目介绍一下，为什么选择它?
3. 运行内核、业务扩展和服务入口如何分层?
4. Durable Checkpoint 具体保存了什么?
5. 这部分是自行实现的，还是直接使用 LangGraph 的快照?
6. 为什么只用 LangGraph 承载核心循环，而不是把全部运行机制交给它?
7. 工具调用怎样保证幂等?
8. 追问：调用时连接断了，但不知道外部操作是否成功，这种情况怎么办?
9. 幂等和可靠性由 Harness 保证，还是由具体业务工具保证?
10. 运行租约和 Fencing Token 分别解决什么问题?
11. Token 为什么需要单调递增，由谁发放?
12. Worker 写状态时怎样校验 Token，是否使用 CAS?
13. 平台的节点抽象是什么样的?
14. 举一个比较复杂的节点，说明它如何执行。
15. 业务数据、逻辑和动作如何接入 Workflow 或 Agent?
16. 多个任务共享同一张执行图时，运行状态怎样隔离?
17. 动态 DSL 每次编译为图，会不会影响性能?
18. Agent 如何持续做评测?

19. 测试用例、参考轨迹、任务完成度和 Judge 怎么配合?
20. 如果主 Agent 跳过了必须执行的规则检查，却直接完成发布，怎么发现和阻止?
21. 如何声明必须调用的工具、必须生成的产物?
22. 最后的校验没通过时怎么处理?
23. 将 Workflow 封装为 Skill，会不会限制 Agent 自由度，又带来什么成本?
24. 大量运行中任务需要迁移时，怎样处理?
25. 追问：如何避免任务丢失或一直卡住?
26. 如果同时有独立复现和团队项目，如何区分各自覆盖的能力与个人贡献?
27. 如何比较 LangGraph、AgentScope、Google ADK 等框架?

本轮小结

这轮明显深入到了分布式可靠性，覆盖 Checkpoint 的保存内容、工具副作用状态未知、运行租约、Fencing Token、CAS、图定义复用与运行状态隔离。另一条主线是强约束任务：必要步骤被跳过怎么办，完成产物如何校验，高吞吐下怎样避免漏处理和卡住。

追问会细到 Token 由谁生成、是否递增、Worker 写状态时怎样校验，以及工具超时后由谁确认外部操作结果。后面又用流程约束检验 Agent 是否真的完成了任务，而不只是返回一段答案。准备时最好画故障时序，明确每次读写和所有权检查的位置；同时解释哪些步骤可以自主规划，哪些必须经过确定性校验，以及这样做的代价。

五、美团

一面

1. 选一个最有代表性的项目，说明解决的问题、业务价值和个人职责。
2. 为什么划分为通用运行内核、业务扩展和服务入口?
3. 业务扩展模块与 Skill 是什么关系?
4. 除了声明文件或 YAML 配置，还包含哪些真正的运行逻辑?
5. 用代码实现的 Workflow，怎样装配进运行时?
6. Tool 和 Skill 数量增加后，上下文怎么控制?
7. 不同业务域的 Sub-agent，能力和权限怎样划分?
8. 渐进式披露具体怎样实现?
9. 这些设计与标准 ReAct Loop 或现有开源 Agent 有什么不同?
10. 你主要负责运行稳定性，还是也负责最终输出质量?
11. Workflow 什么时候接管执行，执行后如何归还控制权?
12. 触发条件是固定规则，还是由模型决定?
13. 自然语言输入、页面操作和显式选择 Skill，是否走同一条链路?
14. 必须执行某个 Skill 时，怎样保证模型不会选错或跳过?
15. 没有调用必需工具时，系统能否发现和补救?
16. 举一个真实 Bad Case，说明原因、修复过程和结果。
17. 追问：除了修改 Prompt，还采取了什么工程措施?
18. 跨多轮对话使用同一个 Skill，会不会反复注入相同内容?
19. 哪些消息会持久化到历史记录里?
20. 追问：不希望保存的内容在哪一步过滤，数据模型怎样区分?
21. 创建一个结果与修改已有结果，是否分别处理?
22. 用户提出超出能力范围的请求时，系统怎么回应?
23. 普通文本回答与结构化 Artifact 如何区分?
24. 如果用户要求修改 Artifact 中不支持的字段或布局，怎么办?
25. 多租户隔离在哪些层实现?
26. 沙箱解决什么问题?
27. 工具幂等与分布式锁是什么关系?
28. 超时、重试、降级和错误返回怎样处理?
29. 每次任务是否都有独立工作空间?
30. Agent 从一个业务域访问另一个业务域时，权限怎样控制?
31. 当前实际使用规模如何?
32. 为什么不直接用通用 Agent，加行业专家和同一套 MCP 工具?
33. 追问：在相同模型、工具和任务条件下，自研方案到底好在哪里?

34. 如果通用 Agent 效果更好，客户为什么还要为自研方案付费?

35. 怎样避免最终变成高成本的定制开发?

36. 探索型任务和稳定的生产任务，是否应该采用不同方案?

37. 产品或解决方案角色如何参与定义价值，而不是只由研发判断?

本轮小结

技术范围覆盖 Skill 选择、上下文持久化、Workflow 调用、结构化产物、工具错误和多租户隔离，但整轮有一个持续出现的问题：这些工程工作最终增加了什么价值？面试官从实现细节一路追到通用 Agent 对照、客户为什么付费，以及定制交付能否复用。

其中，Skill 内容是否跨轮次保留、漏调工具怎样发现、超出产物能力的请求怎样处理，都要求讲到具体数据和执行规则。自研价值的讨论又要求在相同模型、工具和上下文下比较，而不是只列额外模块。准备时应同时有一个能解释实现的 Bad Case 和一个能说明价值的对照案例，区分效果、可靠性、权限与交付成本各自带来的收益。

六、拼多多

一面

1. 请做一下自我介绍。
2. 项目是否已经落地，实际使用规模如何?
3. 怎样区分 Agent Harness 与 Agent Framework?
4. 运行内核包含哪些主要模块?
5. 项目主要面向 ToB 还是 ToC?
6. 通用运行层与业务能力如何分层?
7. 多个任务共享 Worker 时，状态怎么管理?
8. 为什么同一时刻只激活一个 Skill?
9. Workflow 与 ReAct 如何衔接?
10. 不同行业的业务差异怎样处理，能否不修改内核?
11. 共用工作空间会不会导致文件或数据串到其他任务?
12. 沙箱如何设计?
13. Docker、Kubernetes 或 Serverless 分别适合什么情况?
14. 文件、网络和系统权限如何限制?
15. Skill 选择正确与否，如何评估?
16. 主 Agent、专用 Agent 和 Sub-agent 如何配置?
17. Memory 由谁决定写入，什么时候写?
18. 记忆存在哪里，怎样向量化和检索?
19. 记忆只是不断追加吗，怎样更新和删除?
20. 新旧记忆发生冲突怎么办?
21. 低价值记忆怎样遗忘?
22. 追问：如果存进去的是错误记忆，怎样纠正?
23. 工作记忆、短期记忆和长期记忆如何区分?
24. 多 Agent 相互交接时，会不会出现循环调用?
25. Handoff 应传哪些结构化信息?
26. 进程重启后，Checkpoint 如何恢复?
27. 追问：父任务和子任务存在依赖时，怎样恢复它们的关系和执行顺序?
28. 工具已经产生副作用，恢复后如何避免重复操作?
29. 完整 RAG 链路如何设计?
30. 如果记忆规模很小，还有必要使用向量库和 Reranker 吗?
31. 怎样看待 grep、glob、PageIndex 等不同检索方式?
32. 网页内容中带有 Prompt Injection，怎么办?
33. 信任边界、最小权限和人工确认应如何配合?

34. Thread.sleep(0) 有什么作用?

35. **【算法题】** 给一个含重复元素的数组，重新排列，使相邻元素不相同。

36. **【算法题 · 追问】** 什么情况下无解，如何判断?

37. 平时有什么兴趣，最近读了什么书?

本轮小结

范围包括 Harness 设计、共享 Worker、Skill、Memory、多 Agent 恢复、检索和安全，并穿插 Java 基础与数组重排算法。Memory 是连续追问较多的一部分，从写入和检索，问到更新、删除、冲突、纠错和淘汰，要求考虑长期使用后的问题。

安全与恢复也不只看单次调用：共享工作目录会不会泄漏文件，网页指令是否可信，两个 Agent 互相转交怎样停下来，父子任务中断后怎样继续。准备时可以按记忆生命周期和任务生命周期分别梳理。算法题则要兼顾构造方法与无解条件，不能只给出某个样例的排列；Java 题属于基础语义考察，要与 Agent 架构准备分开练。

七、腾讯

一面

1. 请做一下自我介绍。
2. 项目有哪些模块，一次请求的执行流程是什么?
3. 系统怎样部署，最终由谁使用?
4. 上下文预算如何分配?
5. 历史消息和工具结果分别如何压缩?
6. 压缩丢掉的信息后续又需要了，怎么找回来?
7. 在什么时候触发压缩？同步做还是异步做?
8. 是否支持多模态输入，整体链路如何处理?
9. 视觉模型与文本模型什么时候切换，由谁决定?
10. PDF、Word 等文档怎么处理?
11. 历史数据与实时信息如何结合，更新怎样生效?
12. 用户手工编排 Workflow，与自然语言生成计划有什么区别?
13. DSL 如何转换为执行图?
14. 视频生成等耗时较长的工具，怎样放进 Agent 执行流程?
15. 计划保存在哪里？步骤失败、中断后怎样继续?
16. 异步结果通过轮询还是回调拿到?
17. 追问：拿到结果后，怎样恢复原来的任务?
18. Human-in-the-loop 在什么时候介入?
19. 用户补充输入、修改方向和等待确认，分别怎么处理?
20. 恢复执行时，如何确定输入对应哪个任务、哪个等待点?
21. RAG 链路如何设计?
22. Excel 或大型表格怎样解析和检索?
23. **【算法题】** 二叉树的层序遍历。题型来自面试后的补充回忆，完整题面及作答过程未能确认。

本轮小结

这轮主要围绕用户交互中的运行问题，包含上下文压缩、多模态输入、模型切换、文件处理、SSE 重连、外部异步任务和人工确认。视频生成一类任务被用来讨论计划如何保存、步骤如何执行，以及外部完成通知怎样触发续跑。

比较集中的追问是不同状态怎样衔接：历史消息补发与实时事件怎样接上，外部任务结束后继续哪一步，用户回复怎样找到对应中断点。上下文部分也会问压缩何时发生、同步还是异步。准备时适合画出前端连接、后台任务和外部服务各自的时序，避免把恢复运行状态当成恢复消息流。二叉树层序遍历另属算法题，来源是补充回忆，不据此判断现场完成情况。

八、蚂蚁

一面

1. 请做一下自我介绍。
2. 如果参与过 Java 微服务项目，主要负责哪些模块?
3. 以实时通信系统为例，底层采用 SIP、VoIP 还是其他协议，应如何选择?
4. 通信控制层与上层业务服务分别负责什么?
5. 系统的坐席和并发规模如何?
6. 连接状态由底层通信组件还是上层调度服务维护?
7. 怎样保证通话状态同步和准确?
8. 状态事件通过消息队列还是回调传递?
9. 如果参与过数据处理平台，平台解决什么问题，个人负责哪部分?
10. 以设备数据为例，怎样设计上报和存储链路?
11. 使用什么时序数据库?
12. 如果数据上报丢失，怎样发现和定位?
13. 用户怎样配置指标计算逻辑?
14. 流式数据的时间窗口如何实现?
15. 追问：为什么没有采用滑动窗口?
16. 为什么需要中间汇总计算层?
17. 指标定义发生变化后，历史数据怎么重算?
18. 追问：如果重新设计指标版本与重算机制，会怎样改进?
19. 是否考虑过使用 Redis 承载中间计算?
20. 为什么选择 LangGraph?
21. LangGraph 在平台中具体承担什么职责?
22. 为什么不用封装程度更高的 Agent 框架?
23. 从文件上传到问答，知识库的完整处理链路是什么?
24. 超大型文件或十几万行的 Excel 如何处理?
25. 关联信息在文档中相距很远，怎样保证一起召回?
26. Metadata Filter 在检索中怎么使用?
27. PPL 语义边界检测如何实现?
28. 追问：这种切分方案有哪些问题?
29. 如果 Recall@5 突然大幅下降，怎么排查?
30. 追问：用户、场景和配置都没有变化，应优先怀疑什么?
31. Embedding 是否“幂等”，同一输入能否得到稳定输出?
32. 追问：怎样验证输出的稳定性?
33. 使用什么 Reranker?

34. Reranker 的输出是否稳定?
35. 追问：如何保证模型排序结果符合格式和行为约束?
36. Agent 执行链路如何做到可观测?
37. 追问：有了可观测性，怎样进一步做到可评测?
38. 最终输出与工具调用轨迹分别怎样评价?
39. 为什么独立建设 Agent Harness?
40. 通用运行内核、服务入口与业务扩展怎样分工?
41. 如何约束业务实现，避免侵入运行内核?
42. Codex 与 Claude Code 有什么区别?
43. 追问：行为差异主要来自模型，还是 Harness?

本轮小结

这一轮兼顾传统后端与 Agent 工程，前面问事件状态同步、数据丢失、时间窗口和指标重算，随后进入 LangGraph、文档检索、模型输出稳定性、可观测性与评测。后端部分并非无关铺垫，它同样在检查数据流转、异常定位和历史数据处理的理解。

RAG 里较有区分度的是“场景和配置看起来没变，召回率为什么突然下降”。这里需要给排查顺序和验证办法，而不只是列原因；Embedding 与 Rerank 的稳定性也不能凭印象判断。准备时可把数据、模型、索引和配置分别列为检查对象，并说明如何做重复测试和对照。观测到评测的追问，则要求区分运行记录与结果质量。

二面

1. 请做一下自我介绍。
2. 业务 Agent Harness 是当前的重点工作吗?
3. 项目团队的开发人员和其他角色怎样配置?
4. 主要采用 AI 辅助编码，还是传统人工编码?
5. 你在项目中具体负责什么?
6. 多人同时使用 AI Coding 时，怎样拆分任务、控制协作和集成风险?
7. 业务扩展模块是否按业务域划分?
8. 运行内核由多少人维护?
9. 你在团队中的职级和角色是什么?
10. 不同用工类型的人员怎样分工?
11. 运行内核、业务扩展和服务入口是独立 REST 服务，还是装配在同一个服务内?
12. 运行内核从零实现，还是基于开源项目?
13. LangGraph 在其中承担什么职责?
14. 一轮 ReAct 包含哪些节点?
15. Observation 里记录哪些信息?
16. 为什么可以把这套运行内核看作最小 Harness?
17. 业务扩展模块具体包含什么?
18. 追问：它与 Skill 是什么关系?

19. MCP、外部业务系统适配和权限约束放在哪一层?
20. 描述式 Skill 和 Workflow Skill 有什么区别?
21. 确定性 Workflow 如何挂起并恢复 ReAct?
22. 目前经过多少真实客户的验证?
23. 为什么考虑换工作?
24. 目前的职级、绩效和薪酬结构如何?
25. 应用平台与 Dify、Coze、Palantir 分别是什么关系?
26. 下层业务数据、逻辑和动作怎样被应用平台使用?
27. 平台当前有多少客户和线上用户?
28. 如果没有深度业务共建，平台怎样形成超越通用低代码产品的竞争力?
29. 低代码产品本来是为了降低门槛，为什么仍需要研发人员陪同客户落地?
30. 什么情况下需要从画布平台转向 Agent Harness?
31. 一次客户交付之后，哪些东西能够成为长期资产?
32. 原平台的 Workflow 和 Agent 能否迁移到新运行时?
33. 最复杂的 Workflow 大概有多大?
34. 平台支持哪些节点类型?
35. 迁移后，原有 Workflow 能力怎样继续承载?
36. 你说的 Workflow Skill，究竟是描述文件，还是一段可执行代码?
37. 追问：业务逻辑是不是仍然需要研发人员编写代码?
38. 运行内核以外，还有哪些能够稳定复用的公共资产?

本轮小结

这轮覆盖运行内核与开源框架的分工、ReAct 节点、业务扩展、MCP 接入、权限以及 Workflow 式 Skill，也讨论了 AI 辅助开发下的协作和平台的复用价值。相比一面较多的数据处理与排障，这轮更关注系统由谁开发、各部分怎样组合，以及长期能留下什么通用能力。

Skill 的概念被反复澄清：到底是描述文件还是代码，怎样执行，为什么需要挂起和恢复主循环。平台价值也不能只靠“支持低代码”来说明，还要解释为什么仍需研发参与、跨场景能复用哪些部分。准备时应明确配置、协议、代码和服务之间的关系，并能举出从业务需求到能力接入的完整过程。

九、百度

一面

1. 请做一下自我介绍。
2. 是否了解百度千帆 AppBuilder?
3. 是否了解 Coze、Dify 等 AI 应用开发平台?
4. 当前平台的产品定位和技术选型是什么?
5. 为什么使用 LangGraph，平台自行实现了哪些部分?
6. 画布式 Workflow 编排中最大的难点是什么?
7. 大型复杂流程如何通过子 Workflow 拆分?
8. 主流程与子流程怎样传递上下文和结果?
9. 追问：子流程出现中断或异常，主流程如何处理?
10. 代码节点怎样保证执行安全?
11. Docker 或 Kubernetes 提供了哪些隔离机制?
12. 从文件上传到生成回答，完整 RAG 链路是什么?
13. Excel 等结构化文档如何解析和切分?
14. 固定切分、层级切分和语义切分如何选择?
15. PPL 语义切分的原理和依据是什么?
16. 追问：怎么通过评测证明这种切分有效?
17. BM25、向量检索、RRF 和 Rerank 如何组合?
18. 答案需要的信息跨多个 Chunk 时，怎样召回?
19. 怎样看待 Coding Agent 使用 grep 或文件遍历，而不是向量 RAG?
20. PageIndex 一类层级索引适合什么场景?
21. 流式节点与非流式节点如何衔接，并统一展示?
22. 追问：是否研究过 Dify 对这两类节点的抽象?
23. 平台的客户量、调用量和并发规模怎样?
24. 平台如何部署，Pod 和 Worker 如何组织?
25. Python GIL 对多核并发有什么影响?
26. FastAPI 的并发依靠协程、线程还是进程?
27. 线上接口延迟突然升高，怎么排查?
28. PostgreSQL 慢查询怎么定位，EXPLAIN 和索引如何使用?
29. **【口述算法题】** 怎样实现固定容量的 LRU Cache?
30. **【口述系统设计题】** 如果自己实现线程池，需要哪些核心组件，执行流程是什么?
31. 追问：任务进入队列之后，由谁驱动 Worker 取出并执行?
32. 如果项目涉及 Ray，会用它承担哪些工作?
33. Ray Head 与 Worker 如何解耦?

34. Head 故障后，任务调度、结果汇聚和后续任务怎样继续?

本轮小结

这是一轮 Agent 技术与后端基础结合得比较多的面试。前半段看主子 Workflow、异常传播、代码节点安全、RAG 和流式节点，后半段转向 Python 并发、慢接口、PostgreSQL、LRU、线程池和调度节点故障。对检索既问策略，也问如何用评测证明有效。

重点不只是会使用组件，还包括主子流程如何传状态、接口慢时先查什么、调度节点退出后哪些工作受影响。LRU 与线程池在本次以口述方式考察，需要解释数据结构、任务流转和并发处理。准备时可以分别练完整调用链、排障顺序和基础实现；GIL、协程、线程、进程的作用尤其要讲准确，不能用“支持异步”概括全部并发机制。

二面

1. 研发经历是否都与 Agent 有关？如果涉及方向转型，是在什么阶段?
2. AI 应用平台具体是什么，客户是谁?
3. 部署在哪里，目前规模多大?
4. 你在平台中具体承担哪些工作?
5. 为什么选择 LangGraph，它在系统中负责什么?
6. 工具分层最终解决谁的问题？模型看到的不都是工具吗?
7. 用户创建的 AI 应用如何评价，有哪些具体指标和数据?
8. P95 TTFT 统计什么阶段？首响应较长时应如何分析原因?
9. 画布平台、Agent Harness 和客户交付是什么关系?
10. 客户最终购买什么，拿到什么交付物?
11. 客户数据在这条链路中如何处理?
12. 平台的执行引擎与独立 Agent 运行内核，本质区别是什么?
13. Skill 有什么作用?
14. 追问：为什么还要把 Workflow 包装成 Skill?
15. 企业 Agent 为什么不像 Codex、Claude Code 那样保持更开放的自主性?
16. App 关闭再打开后，怎样继续看到原任务的流式输出?
17. 追问：SSE 重连怎样做到不漏字、不重复，协议具体怎么设计?
18. Workflow-as-Skill 中是否包含外部 I/O?
19. 追问：这样一个 Workflow 与普通 API 服务有什么区别?
20. Human-in-the-loop 在哪里中断，如何恢复?
21. Redis 中的状态过期以后，任务还能继续吗?
22. 上下文工程具体做了哪些事情?
23. 项目是否包含 Memory，什么应该记、什么不应该记?
24. “支持同步、异步”具体指什么?
25. 追问：是否存在两套不同的 ReAct Loop?
26. 现在是否还独立写代码，能否现场完成一道简单 Coding?

本轮反问后还有现场编码。题目通过屏幕展示，现有信息不足以还原完整题面、输入输出和最终代码，因此不补写具体算法题。

本轮小结

这轮多次要求澄清已有表述的具体含义：工具分层解决什么问题，平台 Runtime 与 Agent Runtime 有何不同，Workflow 式 Skill 与普通 API 调用差在哪里，所谓同步与异步究竟指什么。性能指标和交付形态也会被拿来反复核对。

技术上较深入的是 SSE 续传、人工中断恢复和缓存失效后的任务处理。恢复了后台状态，是否就能保证前端不漏消息、不重复消息，需要另外解释；人工回复如何关联中断点同样不能省略。准备时建议用实际接口、消息顺序和存储位置说明。末尾还有编码环节，但题面无法可靠还原，因此这里没有补写具体算法题。

三面

1. 介绍一下当前工作，重点讲技术设计、遇到的问题和解决方式。
2. Harness 怎么设计，运行过程是什么，如何提升效果?
3. 通用运行内核、业务扩展和服务入口怎样分工?
4. 实际交付给客户的是中间件，还是云上的产品?
5. ReAct、Skill 和 Browser Use 等能力运行在哪里?
6. 为什么多个 Agent 共享 Worker?
7. 不同用户和业务数据怎样隔离?
8. 是否提供代码和 Bash 能力，代码在哪里执行?
9. 为什么使用 LangGraph，而不是直接实现一个 Loop?
10. 运行图包含哪些节点?
11. 追问：LangGraph 能否替换，替换会影响哪些部分?
12. Skill 中的代码怎样运行?
13. Workflow 形式的 Skill 如何定义和执行?
14. LangGraph Checkpoint 与自行建设的持久化恢复机制，有什么区别?
15. 如何评估性能和稳定性?
16. 如果一次工具调用失败，但最后任务成功了，成功率应该怎么算?
17. Agent 的 Ground Truth 和测试用例如何构建?
18. 追问：怎样判断任务真正完成?
19. 耗时、性能和 Token 消耗目前是什么水平?
20. 追问：如何判断首个可见响应时间是否过长，统计的究竟是哪段时间?
21. 实际客户、终端用户和并发规模如何?
22. BM25、Embedding、RRF、Reranker 与 MMR 怎么组合?
23. 如果要继续提高召回率，最难的部分是什么?
24. RAG 评测是否亲自做过，用过哪些指标?
25. Recall@K、MRR、NDCG 分别适合衡量什么?
26. 是否关注 Codex、Claude Code、WorkBuddy 等新方案?

27. 是否阅读源码，最近看到了哪些设计变化?

28. 你提到的新 Harness 有什么优势？这种优势是绝对的吗?

29. 为什么现在考虑换工作?

本轮小结

这轮主要复核架构选择和工程指标，范围包括共享 Worker、用户数据隔离、代码执行、LangGraph 的可替换性、Checkpoint、任务评测和 RAG 指标。它与二面都重视表述是否准确，但更集中地问为什么这样设计，以及如何判断设计后的系统表现。

评测是需要重点准备的部分：工具失败但任务成功怎么统计，Ground Truth 怎样构建，Recall@K、MRR、NDCG 各看什么，延迟和 Token 消耗又怎样计量。技术视野的讨论也要求说明某个 Harness 的优势在哪些条件下成立。准备时应把运行健康度、任务完成度和答案质量分开，不用单一成功率概括所有结果。

十、深信服

一面

1. 请做一下自我介绍。
2. 最近的工作主要都与 Agent 有关吗?
3. 具体涉及平台研发、框架开发还是业务应用落地?
4. 选一个项目，讲讲怎样快速落地 Agent 应用。
5. 刚才业务场景讲得比较多，能否进一步介绍 Agent 架构设计?
6. 框架更偏固定 Workflow，还是自主规划 Agent?
7. 在 Agent 的具体设计中，你承担了哪些工作?
8. 参考通用 Coding Agent 的实现思路，当前业务 Agent 还有哪些可以改进的地方?
9. 如何将具体业务场景和固定 Workflow 加入 ReAct 执行过程?
10. 追问：有哪些具体参考例子?
11. 是否有后端或平台研发经验？如有，涉及哪些方面?
12. 是否有 NLP 相关研究经历?
13. 工作中是否参与过大模型训练?
14. 当前 Agent 项目涉及模型训练，还是主要调用模型 API 或本地部署模型?

本轮小结

这一轮先从应用如何落地切入，但面试官随后明确把话题拉回 Agent 架构，继续问固定 Workflow 与自主规划的区别、ReAct 如何接入业务流程，以及可以参考哪些通用 Agent 的设计。也涉及应用研发、模型调用或部署、模型训练之间的经验范围。

从提问的转向看，业务介绍需要服务于技术解释：场景交代清楚后，要尽快说请求如何执行、哪些步骤固定、哪些由模型决定、工具和记忆怎样参与。准备时可以用一张简图讲完系统，再挑一项实际设计深入说明。对模型相关问题，也要明确讨论的是应用、部署还是训练，不把它们笼统归为大模型研发。

十一、同花顺

一面

1. 近期的主力开发语言是什么？如使用过 Python 或 Java，分别用于哪些场景?
2. 公司内部的技术栈是否受到合规等限制?
3. 团队使用哪些 AI Coding 工具和模型?
4. AI 应用平台是什么产品，解决什么问题?
5. 已有很多低代码 AI 平台，为什么仍然选择自研?
6. 平台是否上线，用户量、调用量和并发量如何?
7. LangGraph、FastAPI 和 Python Worker 为什么能够支撑商业化平台?
8. 如果 Workflow 没有 LLM 节点，而是大量代码节点，应该怎样执行?
9. 追问：代码中出现死循环怎么办?
10. ReAct 的核心执行逻辑是什么?
11. 以查询天气、控制电视和空调为例，从用户输入到 Function Call、Observation 和 Final Answer，完整链路怎样运行?
12. 工具数量增加后，怎样避免选错工具或参数错误?
13. 渐进式披露具体怎样实现?
14. 几十个 Skill 如何被发现和激活?
15. Skill 同时包含 Markdown、脚本、代码和工具时，运行时怎样执行?
16. Workflow-as-Skill 如何与 ReAct 主循环衔接?
17. 为什么有了 MCP 之后，还需要 Skill?
18. 反过来，有了 Skill，MCP 能不能完全不用?
19. Skill 直接调用 OpenAPI、CLI 或临时生成代码，为什么不能替代 MCP?
20. 追问：如果 API Key 已在环境变量中，模型只需要执行简化命令，MCP 还提供什么额外价值?
21. MCP 相比标准 OpenAPI 真正增加了什么?
22. 你提到的安全、治理和复用价值，是 MCP 独有的吗?
23. 面向千万用户的云端 Agent，整体怎样设计?
24. 是否需要给每个用户运行一个独立 Agent 实例?
25. 用户自定义 Skill、长期记忆、偏好和上传文件如何加载、隔离?
26. 公共资产与租户私有资产如何做到物理复用和逻辑隔离?
27. 用户要求执行 ps -ef 或终止某个 Java 进程时，如何保证安全?
28. Agent 在沙箱内运行与在沙箱外调度，架构有什么不同?
29. 云端 Agent 的性能、资源成本、隔离和安全如何权衡?
30. **【Python 编码题】** 编写一个协程示例。
31. **【Java 编码题】** 给出两种单例实现方案。
32. **【Java 编码题 · 追问】** 双重检查锁为什么需要 volatile，锁应该加在哪里?

33. 平时是否在本机使用 Codex 等 AI Coding 工具?

34. 项目代码由谁编写，AI Coding 占多大比例?

本轮小结

范围从平台自研理由、ReAct 与 Function Calling，扩展到 Skill 发现和执行、MCP 替代关系、云端多租户、工作空间和沙箱，最后还有 Python 与 Java 编码。Skill 与 MCP 的讨论比较集中，面试官会用 CLI、OpenAPI 等替代方式继续追问，要求解释各方案的差异和实际价值。

云端系统设计则改变了原问题的规模，需要重新考虑用户状态、公共与私有资产、资源成本和危险命令的执行位置，不能直接套用小规模部署方式。准备时先确认场景和隔离边界，再讨论共享或独占资源。协程与单例属于语言基础编码题，分别检查异步用法和线程安全，不应与前面的架构讨论混为一项。

十二、编码题汇总

来源	类型	题目	准备重点
字节一面	算法题	Two Sum 类问题	哈希表、下标与重复元素、复杂度；完整题面信息不全
字节二面	算法题	原地合并两个有序数组	预留空间、指针方向、空数组与剩余元素
字节三面	工程编码题	ReAct Loop 伪代码	完整主循环、终止条件，以及 Skill / Memory 的扩展位置
淘天一面	工程编码题	Plan-and-Execute Agent	计划表示、步骤执行、前序结果引用、工具与模型步骤的区分
拼多多一面	算法题	重排数组，使相邻元素不相同	频次分布、可行性条件、构造方法与边界
腾讯一面	算法题	二叉树层序遍历	队列、逐层处理、空树；题型来自补充回忆
百度一面	口述算法题	固定容量 LRU Cache	查询、更新、访问顺序与淘汰
百度一面	口述系统设计题	线程池	任务队列、Worker 调度与并发控制
同花顺一面	Python 编码题	协程示例	异步代码的组织与执行方式
同花顺一面	Java 编码题	两种单例实现	线程安全、双重检查锁与 volatile

表里的准备重点是复习建议，不是完整题面。拼多多二面未收录，其他未能确认的编码题也没有列入。

十三、几类反复被问到的问题

把各轮放在一起看，有几类问题出现得比较多：

主题	常见追问	对应场次
为什么自研	现有框架缺什么？自己补了什么？用相同模型和工具比较过吗？	千问二面、美团一面、蚂蚁二面
故障恢复	保存哪些状态？崩溃后从哪继续？外部操作会不会重复？多个 Worker 怎么协调？	字节一面、小红书方向2二面、淘天二面
上下文	放哪些信息？什么时候压缩？压缩丢掉的信息还能找回来吗？	千问三面、腾讯一面、淘天一面
Memory	什么时候写？怎么更新、删除和纠错？记忆冲突或者影响了原有规则怎么办？	拼多多一面、小红书方向1一面
Workflow 与 Agent	哪些步骤固定，哪些让模型决定？两者怎么切换？怎么判断任务完成？	淘天一面、美团一面、小红书方向2二面
RAG	怎么解析切分、召回和重排？用什么指标测？效果突然下降从哪里查？	小红书方向1一面、淘天二面、蚂蚁一面、百度三面
权限与隔离	用户身份怎么传到工具层？能看到工具就能调用吗？文件和执行环境怎么隔离？	字节二面、拼多多一面、同花顺一面
指标	具体测的是什么？怎么算的？和什么比较？怎么排除其他因素的影响？	字节二面、千问二面、百度二三面

十四、面试前可以怎么准备

这些面试里，很多问题第一问并不难，难的是后面连续追问。比如回答了“有 Checkpoint”，接着就会问保存什么、在哪一步保存、恢复后工具会不会重复执行。复习时可以顺着这种问法往下准备。

先看清楚岗位在做什么

同样叫 Agent 开发，实际工作差别很大。投递前先看 JD，也可以向招聘方确认具体做什么，再决定复习重点。

- 偏应用开发的，多准备任务拆解、工具接入、RAG 和效果优化。最好能讲清一个需求怎么做完，以及某次效果不好的问题是怎么改好的。
- 偏 Runtime / Harness 平台的，要多花时间在执行状态、扩展接口、Checkpoint、异步恢复、权限和隔离上。画一条正常执行和一条故障恢复的时序，会比较有帮助。
- 偏 AI 后端或基础设施的，并发、队列、数据库、SSE、模型网关、压测和排障都不能落下。性能数字要能解释测试条件和瓶颈在哪。
- 偏 FDE 的，除了写代码，还要讲需求怎么问清楚、已有系统怎么接、部署和权限有哪些限制、最后怎么验收。交付过一次之后，哪些东西能留给下一次用，也值得准备。

训练、微调和推理优化要不要深入，看岗位要求即可。岗位名称里有 AI，不代表这些都要精通；但后端基础题在这批面试里确实出现了不少。

几块需要准备扎实的内容

先把一次 Agent 请求从头到尾讲明白。ReAct、Plan-and-Execute、Workflow 各自怎么执行，什么时候会用 Sub-agent，最好能画出来，也能写出简化代码。Multi-Agent 可以往后放一些，先把单 Agent 的正常执行和异常处理弄清楚，再考虑什么时候确实需要并行分工、独立上下文或不同权限。

Tool、Skill 和 MCP 经常被放在一起问。需要知道它们分别做什么，以及一次能力调用怎么完成发现、加载、参数校验、执行和错误返回。只回答“都可以调用工具”，很容易被继续追问。

上下文和 Memory 可以一起复习，但要分清各自的问题。当前请求放哪些信息、Token 预算怎么分、工具结果太长怎么办；长期记忆什么时候写、怎么更新和删除、旧信息与新信息冲突怎么办，都要有具体说法。压缩做完以后，最好验证一下任务还能不能完成，丢失的内容能不能找回来。

RAG 要熟悉从文档到答案的整个过程：解析、切分、索引、召回、融合、重排和上下文组装。更值得练的是拿一个失败问题排查，判断到底是没切好、没召回，还是召回以后被排掉了。

可靠性部分可以从几个故障入手：任务重复投递、Worker 退出、外部工具结果未知、用户取消、等待人工确认。分别想清楚状态存哪里、谁有执行权、什么情况下能重试。权限和沙箱也要考虑到文件、网络以及外部系统，不能只看模型拿到了哪些工具描述。

评测方面，至少准备一套用例怎么来、参考答案怎么定、轨迹怎么检查、Judge 怎么用、改动后怎么做回归。延迟、成本和成功率都要有明确口径。一次工具失败后任务仍可能成功，工具全成功也可能答错，这几种情况不能混着算。

最后是后端与编码。Python 异步、线程和进程的区别，数据库、缓存、队列和 SSE 的常见问题，以及基础数据结构，建议结合代码复习。

可以动手练几件事

1. 写一个很小的 Agent 执行器。先用模拟模型响应和两个本地工具，跑通调用、Observation 和最终输出，再补参数错误、超时、轮次上限和取消。之后换成真实模型，看看会出现哪些原来没想到的问题。
2. 在不同位置模拟进程退出：工具调用前、工具成功但结果还没保存时、状态保存后。观察重启后会发生什么，再试试重复任务和旧 Worker 恢复。哪些能自动恢复，哪些必须查外部状态或人工处理，都记下来。练习用可重置的模拟数据，不要对真实业务做故障实验。
3. 找一些公开文档，自己确认一组问题和对应证据，做 RAG 对比。每次只改一个主要因素，比如切分方式或重排策略，同时记录失败例子、延迟和成本。否则参数全换了，很难知道究竟是哪一步有效。
4. 限时写代码，练习时先关掉 AI 补全。数组、哈希表、队列、树遍历、LRU 要练，ReAct Loop、Plan-and-Execute 和异步任务也要练。先把输入输出和主流程写对，再补测试与边界。

不用为了准备面试再造一个完整平台。把小例子跑起来，故意弄坏，再解释为什么会坏，就已经能补上不少只看文档时不容易发现的问题。

源码怎么读

选一个和岗位相关的框架，沿一条请求往下读即可，不用一开始就同时看很多项目。先找入口，看会话和运行状态怎么创建，历史消息、配置和身份从哪里加载；再跟到上下文组装、模型输出解析、工具查找和执行。

主流程看懂后，去找退出条件和错误分支：什么情况下进入下一轮，什么情况下结束，预算耗尽、取消和工具报错分别怎么处理。然后看状态在哪里保存、保存失败怎么办、恢复时会重跑哪些步骤。

最后补流式事件和扩展点。前端怎么收到消息，断线后怎么办，Skill、Memory 和业务逻辑从哪里接入。测试也值得一起读，尤其是故障测试，能帮助确认实现有哪些前提、哪些事情并不保证。

读完后试着画一张简单调用图，标出关键函数，再讲一遍正常执行和异常恢复。换一个框架时，拿同一条流程比较就行。面试中如实说读过哪些源码、做过哪些实验，不用把阅读经历说成生产交付经验。

项目介绍怎么准备

先练一个两分钟左右的版本：要解决什么问题，自己负责什么，最后做到了什么。开头不必把所有模块名都列一遍。面试官对某一部分有兴趣，再展开架构、代码和测试。

个人贡献要说具体。哪些方案是自己定的，哪些代码是自己写的，哪些是团队原有能力，哪些直接用了开源框架，都可以讲清楚。还可以准备一个当时没选的方案，说明为什么没选，以及换一种条件后会不会考虑它。

准备一次真实的失败或故障也很有用。从怎么发现问题讲起，说明如何定位、修复和验证，后来加了什么检查来避免再发生。涉及指标时，把样本、比较基线、统计周期和测试环境一起准备好。当前跑了多少实例，不等于系统最多能承受多少并发。

已经上线的、只做过实验的、看源码了解到的、面试时现场推演的，分开说就好。碰到没做过的问题，可以先问清约束再分析，说明哪些判断还需要验证。

时间不多时

优先准备一条能讲清、也能写出简化代码的执行流程；选一个恢复或幂等问题往深处看；整理一个 RAG、工具选择或上下文优化的真实例子。基础算法和工程编码穿插着练，别全部留到面试前一天。

找同伴模拟一轮也很有帮助。不用问很多题，围绕一个设计连续问“为什么这样做”“换个方案行不行”“怎么知道有效”“失败了怎么办”，通常很快就能发现哪些地方还没想清楚。

还有几处容易说得太满：有 Checkpoint，并不代表外部操作就只执行一次；没有向模型展示某个工具，也不能省掉执行层鉴权；Prompt 里的禁止事项，仍需要权限检查来兜住。类似这些结论，最好都能说明条件和限制。